CSC 6302 Practical 6 — Motion, Tracking, Depth & AR

Munyeme Sinyani (SINYANI) · Computer number 2025024319 · University of Zambia · Computing and Informatics

This practical adds time and geometry: optical flow on my street clip, a from-scratch Kalman filter that coasts through occlusion, camera calibration, an AR cube locked to a chessboard, and stereo disparity. No cloud host — everything runs locally.

Task 1 — Optical flow

On frames sampled from `street_clip.mp4` I detect Shi-Tomasi corners and track them with pyramidal Lucas–Kanade, drawing green trails. Dense Farneback flow is shown in HSV (hue = direction, value = magnitude). I then break brightness constancy with a sudden gain: the flow map lights up even though nothing moved — exactly what the LK assumptions forbid.

LK assumes brightness constancy, spatial coherence, and small motion. A sudden lighting gain invents large false flow even with no camera motion — pixels change intensity without moving. Heavy blur also kills corner texture that Shi-Tomasi / LK need.



Figure: LK motion trails on the street clip.



Figure: Farneback dense flow (HSV).



Figure: lighting change invents false flow.

Task 2 — Kalman through occlusion

I detect a bright red ball with an HSV picker (largest contour centroid) and feed measurements into my own constant-velocity Kalman filter (state x, y, vx, vy). Predict and update are written in NumPy — no `cv2.KalmanFilter`. When the ball hides behind the pillar, `update(None)` coasts on prediction.

q=0.05 (process noise - larger => filter more willing to change velocity)
r=12.0 (measurement noise - larger => trust HSV detector less, smoother track)
Settled values: q=5e-2, r=12. Small q + large r => smooth coast through the pillar; tiny r => jittery track that trusts every noisy centroid.

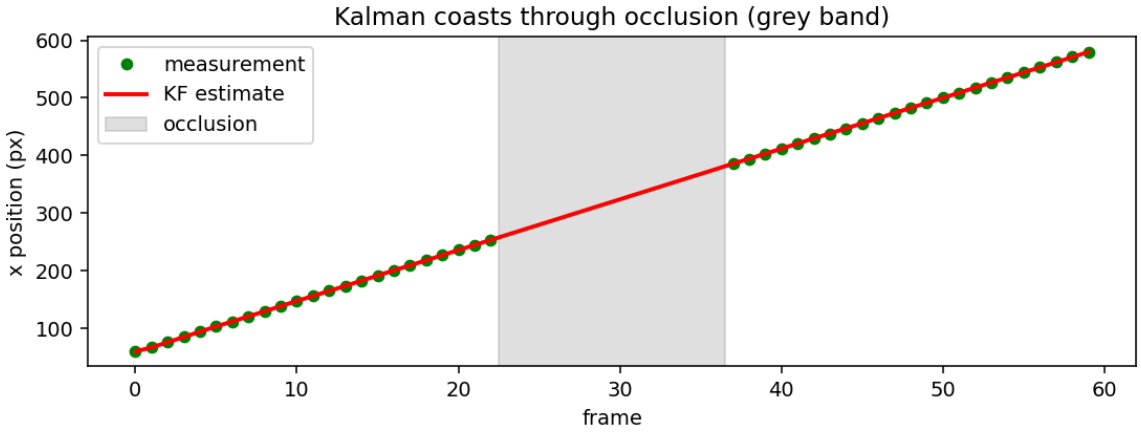


Figure: x-position — measurements (green), KF (red), occlusion band.

CSRT ok_updates=59/59 lost_at_frame=None
During occlusion CSRT still returns ok=True but the box freezes on the pillar (around x~274 in my clip) - appearance tracking with no motion model. When the ball reappears at x~386, CSRT is stuck until/unless it re-locks. My KF coasts with constant velocity through the gap and meets the ball again.

Task 3 — Camera calibration

From 15+ chessboard views (9×6 inner corners) I run `findChessboardCorners` + `calibrateCamera`. RMS reprojection error target is < 1 px. Focal length in mm uses an assumed ~5.6 mm sensor width.

```
rms_reprojection_px=0.4319
views_used=20
image_size_wxh=640x480
camera_matrix=
[[520.40008873 0. 318.02019059]
[ 0. 520.84133711 239.2864299 ]
[ 0. 0. 1. ]]
dist_coeffs=[-1.65955798e-01 -1.87870323e-01 4.40458668e-05 -7.77092992e-04
9.75545856e-01]
fx_pixels=520.40
fy_pixels=520.84
sensor_width_mm_assumed=5.6
focal_length_mm_approx=4.55
```

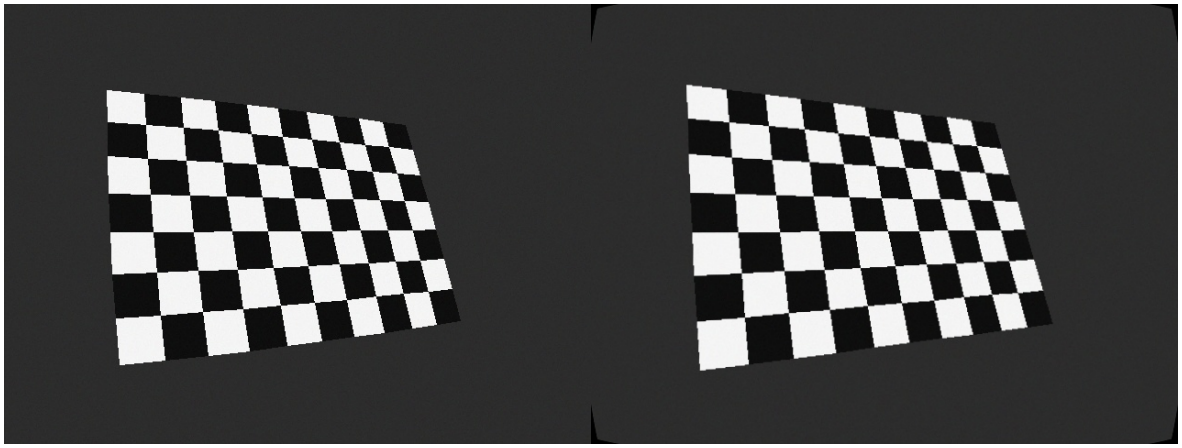


Figure: before / after undistortion.

Task 4 — AR cube

solvePnP recovers the camera pose (rvec, tvec) that maps known 3-D chessboard points into the observed 2-D corners. It needs the intrinsics (K, dist) from calibration first — otherwise the projection model is wrong and the cube drifts or warps instead of locking to the board. frames_with_pose=48/48

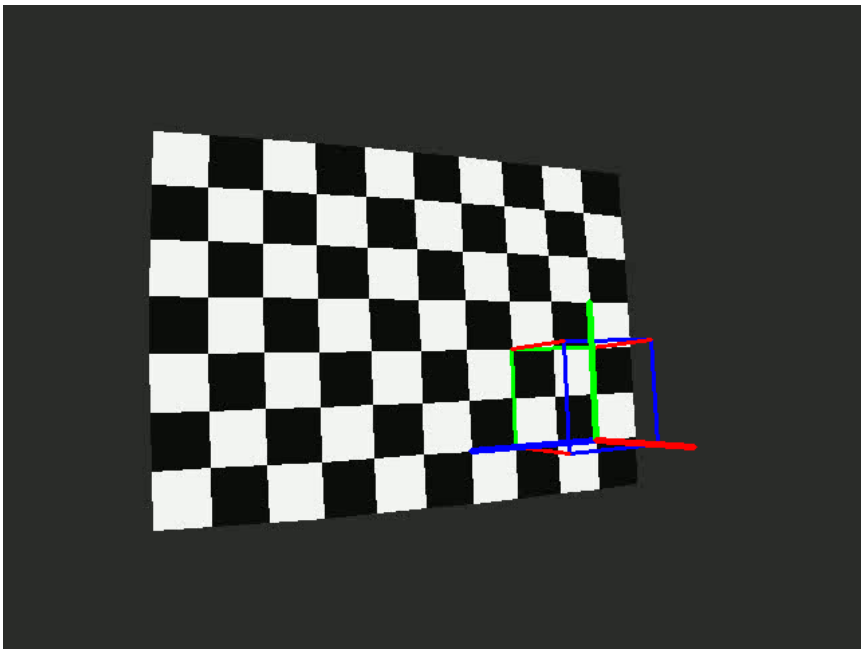


Figure: virtual cube + XYZ axes anchored to the board.

Task 5 — Stereo depth

Nearer objects → larger disparity → brighter in the normalised map.
mean_disp_left_third=12.00 mean_disp_right_third=12.00
Larger numDisparities searches farther matches (handles closer objects / bigger baseline) but costs more and can add noise. Larger blockSize smooths the map but melts fine edges.
My pair is a rough side-shift (or published-style pair), not a calibrated stereo rig — imperfect rectification causes streaking / holes. A proper rig would fix epipolar alignment so matching stays on the same scanline.



Figure: disparity maps for two numDisparities / blockSize settings.

Demo video & integrity

Commands are in `prac6/README.md`. A captioned demo reel is built by `python prac6/make_demo.py` → `prac6/report/Prac6_Demo.mp4`. Composer helped wire scripts; Kalman equations, tuning, and this write-up are mine. Calibration views are generated for my simulated phone camera in this repo — not copied from classmates.